DSA Final LAB-EXAM
CS- 23

Time: 2.5 hours **Total : 50**

Question 1 : Remove Sub-Folders from the Filesystem

Given a list of folders folder, return the folders after removing all sub-folders in those folders. You may return the answer in any order.
If a folder[i] is located within another folder[j], it is called a sub-folder of it. A sub-folder of folder[j] must start with folder[j], followed by a "/". For example, "/a/b" is a sub-folder of "/a", but "/b" is not a sub-folder of "/a/b/c".

- The format of a path is one or more concatenated strings of the form: '/' followed by one or more lowercase English letters.
- For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string and "/" are not..

Example 1:
Input: folder = ["/a","/a/b","/c/d","/c/d/e","/c/f"]
Output: ["/a","/c/d","/c/f"]
Explanation: Folders "/a/b" is a subfolder of "/a" and "/c/d/e" is inside of folder "/c/d" in our filesystem.

Example 2:
Input: folder = ["/a","/a/b/c","/a/b/d"]
Output: ["/a"]
Explanation: Folders "/a/b/c" and "/a/b/d" will be removed because they are subfolders of "/a".

Example 3:
Input: folder = ["/a/b/c","/a/b/ca","/a/b/d"]
Output: ["/a/b/c","/a/b/ca","/a/b/d"]

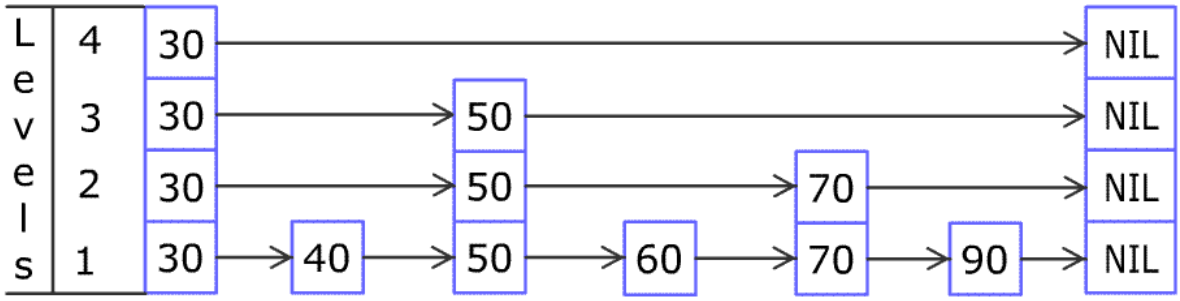
```
class Solution {
public:
    vector<string> removeSubfolders(vector<string>& folder) {

    }
};
```

Question 2: Design Skiplist

Design a Skiplist without using any built-in libraries.

A skiplist is a data structure that takes O(log(n)) time to add, erase and search. Comparing with treap and red-black tree which has the same function and performance, the code length of Skiplist can be comparatively short and the idea behind Skiplists is just simple linked lists.

For example, we have a Skiplist containing [30,40,50,60,70,90] and we want to add 80 and 45 into it. The Skiplist works this way:



You can see there are many layers in the Skiplist. Each layer is a sorted linked list. With the help of the top layers, add, erase and search can be faster than O(n). It can be proven that the average time complexity for each operation is O(log(n)) and space complexity is O(n).

Implement the Skiplist class:
Skiplist() Initializes the object of the skiplist.
bool search(int target) Returns true if the integer target exists in the Skiplist or false otherwise.
void add(int num) Inserts the value num into the SkipList.
bool erase(int num) Removes the value num from the Skiplist and returns true. If num does not exist in the Skiplist, do nothing and return false. If there exist multiple num values, removing any one of them is fine.

Note that duplicates may exist in the Skiplist, your code needs to handle this situation.

Example 1:-

Input:

["Skiplist", "add", "add", "add", "search", "add", "search", "erase", "erase", "search"]
[[], [1], [2], [3], [0], [4], [1], [0], [1], [1]]

Output:

[null, null, null, null, false, null, true, false, true, false]

Explanation:

Skiplist skiplist = new Skiplist();
skiplist.add(1);
skiplist.add(2);
skiplist.add(3);
skiplist.search(0); // return False
skiplist.add(4);
skiplist.search(1); // return True
skiplist.erase(0); // return False, 0 is not in skiplist.
skiplist.erase(1); // return True
skiplist.search(1); // return False, 1 has already been erased.

```
class Skiplist {
public:
    Skiplist() {

    }
    bool search(int target) {
        /////
    }

    void add(int num) {
        ////
    }
    bool erase(int num) {
      ////
    }
};
```